TRƯỜNG ĐẠI HỌC FPT

EMBEDDED LINUX SYSTEMS

BÁO CÁO LAB-01

CẤU HÌNH KERNEL & KHỞI ĐỘNG
HỆ THỐNG LINUX NHÚNG (ARMv7)

TRƯỜNG THÔNG TIN	GIÁ TRỊ
Họ và tên	Nguyễn Trần Nam Nguyên
Mã số sinh viên	SE205174
Nhóm thực hành	Null
Môn học	Embedded Linux Systems
Ngày thực hiện	16/10/2026
Nền tảng	Ubuntu host – QEMU vexpress-a9 – ARMv7 Cortex-A9

Linux Kernel 5.15 • U-Boot 2022.04 • BusyBox 1.35.0 • initramfs • SMP

1. Mục tiêu và môi trường thực hành

Mục tiêu là xây dựng chuỗi khởi động Linux nhúng cho board mô phỏng ARMv7 Cortex-A9, từ việc cross-compile Linux Kernel và U-Boot, tạo rootfs tối giản bằng BusyBox, đóng gói initramfs đến kiểm tra BusyBox shell trên QEMU.

Thành phần	Kết quả đã xác minh
Nền tảng	QEMU vexpress-a9, CPU ARMv7 Cortex-A9
Linux Kernel	5.15.0, ARM zImage, 4,9 MB
Device Tree	vexpress-v2p-ca9.dtb, khoảng 14 KB

Bootloader	U-Boot 2022.04, ELF ARMv7 32-bit, 4,2 MB
BusyBox	ARM EABI5, static linking, stripped
initramfs	initramfs.cpio.gz, khoảng 1,1 MB; gzip -t trả về 0
RAM / CPU thực thi	QEMU -m 512M -smp 2; 2 CPU online

1.1. Cấu trúc tài liệu đối chiếu

Đề LAB-01 (18 trang) chia thành Phần A – Linux Kernel (25 điểm), Phần B – U-Boot (25 điểm), Phần C – BusyBox/initramfs/QEMU (25 điểm), báo cáo & sơ đồ boot (15 điểm), và năm câu hỏi phân tích (10 điểm).

1.2. Phạm vi bằng chứng

Các giá trị cấu hình, kết quả U-Boot, màn hình Kernel Panic và BusyBox/QEMU trong báo cáo được trích từ những lần kiểm tra đã thực hiện. Log cuối quá trình build ban đầu và một phiên đo -smp 1 chưa được cung cấp, vì vậy không trình bày chúng như số đo thực nghiệm.

2. Boot Sequence Diagram – Chuỗi khởi động

POWER ON	Cấp nguồn, đặt trạng thái CPU, bắt đầu trình tự boot
↓	
ROM BOOTLOADER	Mã boot ban đầu tìm và nạp bootloader vào bộ nhớ
↓	
U-BOOT 2022.04	Khởi tạo RAM/thiết bị; chuẩn bị zImage, DTB, initramfs, bootargs
↓	
LINUX KERNEL 5.15	Đọc DTB và kernel command-line; khởi tạo driver, SMP, rootfs
↓	
/init → /sbin/init	Mount /proc, /sys; chạy BusyBox init và /etc/init.d/rcS
↓	
BUSYBOX SHELL	Hiển thị Welcome và dấu nhắc / # để người dùng thao tác
↓	

Dữ liệu bàn giao qua các bước

ROM bootloader chuyển quyền chạy cho U-Boot; U-Boot thiết lập kernel image, Device Tree Blob, initramfs và kernel command-line; kernel khởi tạo userspace; init nạp các dịch vụ và mở BusyBox shell.

Phân biệt mô hình và phép thử: Đây là chuỗi khởi động điển hình của hệ nhúng có bootloader. Trong bài thực hành, QEMU được chạy trực tiếp bằng -kernel zImage -dtb ... -initrd ..., tức QEMU đã nạp kernel thay U-Boot. Phiên U-Boot shell được kiểm tra riêng; không khẳng định U-Boot đã thực sự gọi bootz để khởi động Linux trong lần thử thành công.

3. Phần A – Cấu hình và build Linux Kernel

3.1. Cross-compilation & cấu hình

Target là ARMv7-A với GCC prefix arm-linux-gnueabihf-. File configs/kernel.config trùng hoàn toàn với kernel/linux-5.15/.config (kết quả cmp: CONFIG_MATCH=0).

```
CONFIG_ARCH_VEXPRESS=y
CONFIG_CPU_V7=y
CONFIG_BLK_DEV_INITRD=y
CONFIG_SMP=y
CONFIG_SERIAL_AMBA_PL011=y
CONFIG_SERIAL_AMBA_PL011_CONSOLE=y
```

Tùy chọn	Vai trò
ARCH_VEXPRESS	Hỗ trợ platform ARM Versatile Express
CPU_V7	Hỗ trợ tập lệnh ARMv7
BLK_DEV_INITRD	Cho phép sử dụng initrd/initramfs
SERIAL_AMBA_PL011(_CONSOLE)	Console UART ttyAMA0 trong QEMU
SMP	Kernel có hỗ trợ đa xử lý

3.2. File kernel và DTB

```
output/zImage: Linux kernel ARM boot executable zImage output/zImage: 4.9 MB
output/vexpress-v2p-ca9.dtb: Device Tree Blob version 17 DTB size: 14081 bytes
```

Kernel đã khởi động thành công trên QEMU và uname -a báo Linux 5.15.0 armv7l. Điều này bổ sung bằng chứng thực thi cho file output, thay vì chỉ dựa vào việc file tồn tại.

Hạng mục minh chứng còn thiếu: log nguyên văn của make vexpress_defconfig và những dòng cuối lệnh build gốc. Để có nơi gọi “versatile_defconfig” khi mô tả báo cáo nhưng phần hướng dẫn A.2 dùng vexpress_defconfig; cần giải thích tên defconfig nhất quán theo nền tảng đã dùng.

4. Phần B – U-Boot 2022.04

4.1. U-Boot chạy trong QEMU

```
U-Boot 2022.04 (Oct 16 2026 - 10:17:45 +0700)
DRAM: 512 MiB
```

```
Flash: 64 MiB
Build: 32-bit
Tag_CPU_arch: v7
=>
```

Lệnh version xác nhận U-Boot 2022.04, toolchain GCC ARM 15.2.0. Lệnh bdinfo cho thấy DRAM bắt đầu tại 0x60000000 và dung lượng vùng RAM chính là 0x20000000 = 512 MiB. Flash ánh xạ từ 0x40000000, dài 0x04000000 = 64 MiB.

4.2. Các biến khởi động

```
=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M' => printenv bootargs
bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M

=> setenv bootcmd 'bootz 0x60100000 0x64000000 0x60000000' => printenv
bootcmd bootcmd=bootz 0x60100000 0x64000000 0x60000000
```

Trong chuỗi lệnh của tài liệu, 0x60100000 là địa chỉ kernel, 0x64000000 là vị trí dự kiến cho initramfs và 0x60000000 dành cho DTB. Cần nạp đúng các image tương ứng trước khi chạy bootz. Với U-Boot, khi dùng initrd theo cú pháp bootz chuẩn, tham số ramdisk thường cần chỉ rõ kích thước và kiểm tra cú pháp của bản U-Boot; giá trị trong tài liệu đã được thiết lập chứ chưa chứng minh boot thành công.

4.3. Kết quả saveenv và autoboot

```
=> saveenv Saving Environment to Flash...
Error: start and/or end address not on sector boundary
Failed (1)
```

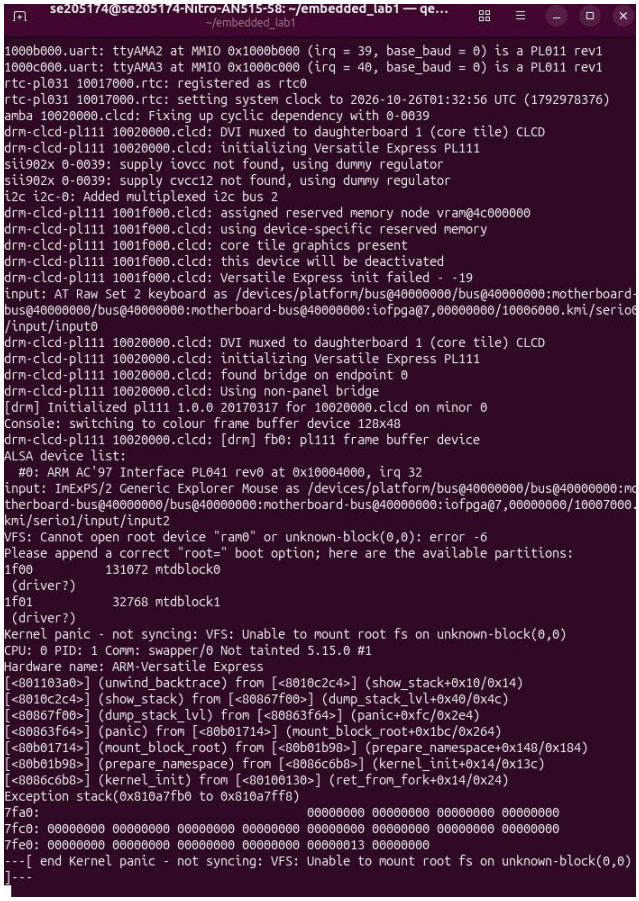
U-Boot đã thử lưu environment nhưng gặp lỗi ranh giới sector Flash; do đó những thay đổi setenv chỉ tồn tại trong phiên làm việc hiện tại. Khi autoboot mặc định, U-Boot cũng từng báo lỗi MMC/TFTP và “Wrong Image Format”; đây là thất bại của đường boot mặc định khi không có image tương ứng, không phải lỗi build bootloader.

5. Thử nghiệm thiếu Root Filesystem – Kernel Panic

Theo mục B.4 của đề, chạy Linux Kernel với DTB nhưng không gắn file initramfs. Kernel không thể tìm thấy root filesystem và dừng khởi động.

```
qemu-system-arm -M vexpress-a9 -cpu cortex-a9 -m 512M \   -nographic -kernel output/zImage \
  -dtb output/vexpress-v2p-ca9.dtb \
  -append 'console=ttyAMA0,115200 root=/dev/ram0 rw mem=512M'
```

```
VFS: Cannot open root device "ram0" or unknown-block(0,0): error -6
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
```



Hình 1. Kernel Panic thực tế khi không cung cấp initramfs (ảnh từ phiên QEMU).

Thông báo thực tế là unknown-block(0,0), khác con số unknown-block(1,0) minh họa trong DOCX. Cả hai đều thể hiện thất bại ở giai đoạn mount rootfs; cần sử dụng nguyên văn log thực tế khi nộp.

6. Phần C – BusyBox và initramfs

6.1. BusyBox static và rootfs

```
rootfs/initramfs/bin/busybox: ELF 32-bit LSB executable, ARM, EABI5,
statically linked
```

Thư mục rootfs có bin/, sbin/, usr/, etc/, dev/, proc/, sys/, tmp/, root/, lib/ và liên kết linuxrc → bin/busybox.
Binary static không cần loader/glibc động trong rootfs tối giản.

6.2. Chuỗi init trong userspace

```
# /init #!/bin/sh mount -t proc none
/proc mount -t sysfs none /sys exec
/sbin/init

# /etc/init.d/rcS mount -t proc none /proc mount -t sysfs none /sys mount -t
tmpfs none /tmp echo "Boot completed! Welcome to Embedded Linux Lab 1"
hostname embedded-lab
```

File /etc/inittab sử dụng ::sysinit:/etc/init.d/rcS và ::askfirst:-/bin/sh. PID 1 chuyển vào BusyBox init, thực thi rcS và mở shell. Việc mount proc/sys ở cả /init và rcS là trùng lặp; phiên kiểm thử vẫn boot thành công.

6.3. Các device node thiết yếu

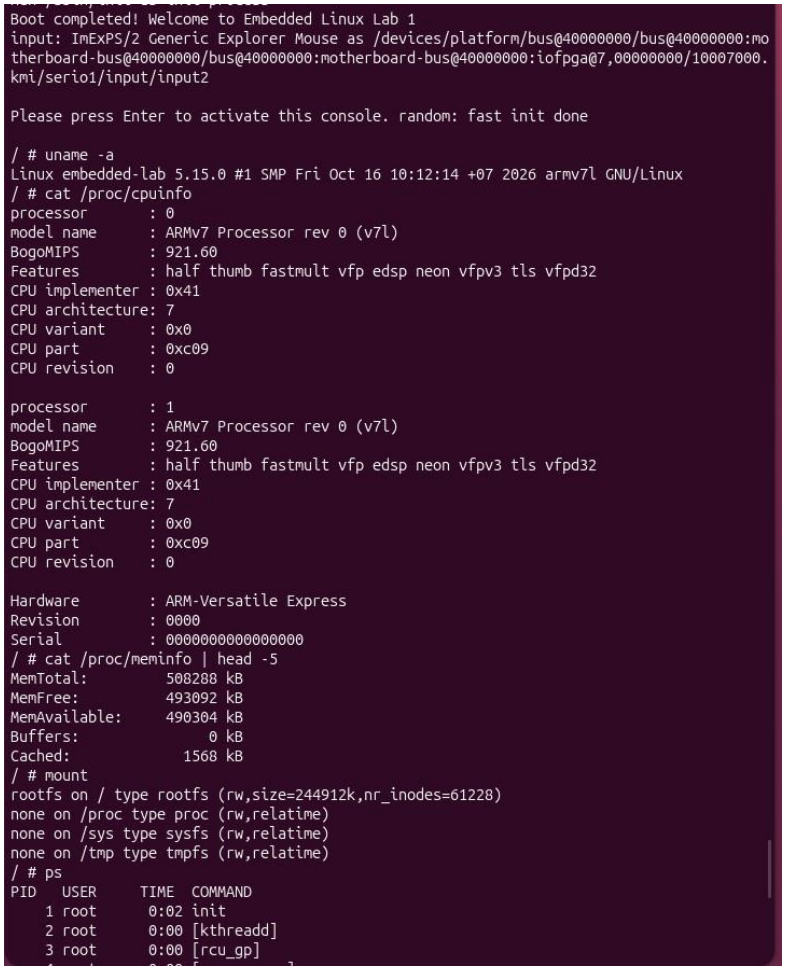
Thiết bị	Major:Minor	Quyền
/dev/console	5:1	600
/dev/null	1:3	666
/dev/zero	1:5	666
/dev/random	1:8	444

File output/initramfs.cpio.gz có dung lượng khoảng 1,1 MB; kết quả gzip -t trả về 0, xác nhận lớp nén hợp lệ. Kết quả QEMU tiếp theo chứng minh initramfs hoạt động thực tế.

7. Boot QEMU thành công và kiểm chứng hệ thống

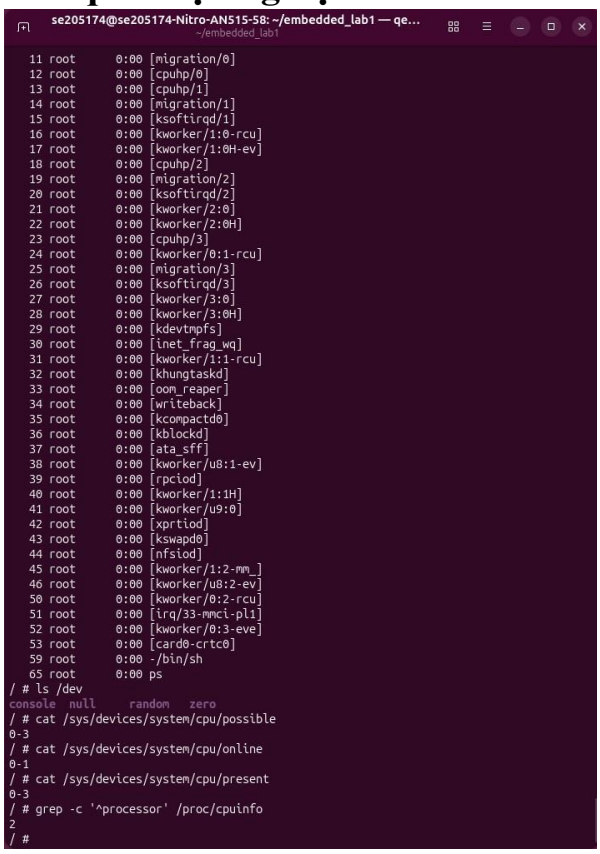
```
qemu-system-arm -M vexpress-a9 -cpu cortex-a9 -m 512M -smp 2 \   -nographic -kernel
output/zImage \
  -dtb output/vexpress-v2p-ca9.dtb \
  -initrd output/initramfs.cpio.gz \
  -append 'console=ttyAMA0,115200 rdinit=/sbin/init mem=512M'
```

Sau khi boot, hệ thống hiển thị Boot completed! Welcome to Embedded Linux Lab 1 và mở BusyBox shell / #.



Hình 2. Linux 5.15 armv7l, Cortex-A9, 512 MiB RAM, các mount point và PID 1 = init.

8. Phân tích SMP và kết quả thực nghiệm



Hình 3. Các CPU possible/present/online và số processor đọc được trên QEMU.

Kiểm tra	Kết quả thực tế	Ý nghĩa
/sys/devices/system/cpu/possible	0-3	Kernel cho phép tập CPU 0 đến 3
/sys/devices/system/cpu/present	0-3	Hệ thống biểu diễn CPU 0–3 có mặt
/sys/devices/system/cpu/online	0-1	Có đúng 2 CPU online
grep -c ^processor /proc/cpuinfo	2	Hai bộ xử lý hiện diện trong cpuinfo

Trên QEMU vexpress-a9, có thể khai báo CPU possible/present 0–3 ngay cả khi chỉ có hai CPU online; số CPU đang chạy cần dựa vào online và /proc/cpuinfo. Như vậy bài thử -smp 2 được xác nhận. Bài so sánh trực tiếp với -smp 1 cần thêm log riêng nếu giảng viên yêu cầu hai ảnh log thay đổi.

9. Trả lời năm câu hỏi phân tích

Câu 1. gnueabi và gnueabihf – ABI và FPU

arm-linux-gnueabi thường dùng chuẩn gọi hàm soft-float ABI: tham số dấu phẩy động đi qua thanh ghi tổng quát/stack theo quy ước ABI, dù mã máy vẫn có thể sử dụng chỉ thị FPU tùy target.
arm-linux-gnueabihf dùng hard-float ABI, truyền tham số float/double bằng thanh ghi VFP theo AAPCS-VFP khi thích hợp. Đối với ứng dụng userspace Cortex-A9 có VFP, hard-float tận dụng ABI tương ứng với thư viện armhf; một thư viện gnueabi và chương trình gnueabihf có thể không tương thích ở ranh giới ABI.

Điểm kỹ thuật quan trọng: Linux kernel không phải ứng dụng glibc thông thường, nên không thể kết luận chỉ vì cross-compile kernel bằng prefix gnueabi thì kernel chắc chắn không chạy trên Cortex-A9. Kernel ARM chủ yếu cần đúng kiến trúc, cấu hình và quy tắc build kernel; sự khác biệt hard/soft-float ABI đặc biệt quan trọng cho chương trình và thư viện userspace. Trong bài đã dùng arm-linux-gnueabihf theo yêu cầu và xác minh kernel ARMv7 boot thành công.

Câu 2. Vì sao DRAM vexpress-a9 ở 0x60000000?

Memory map là quyết định thiết kế của SoC/board được mô phỏng: các vùng thấp của không gian địa chỉ dành cho các thiết bị/bus/flash khác, còn DRAM của vexpress-a9 bắt đầu ở 0x60000000 theo bdinfo. Vì vậy U-Boot phải nạp zImage, DTB và initramfs vào vùng RAM hợp lệ, không chồng lấn nhau và không ghi đè vùng U-Boot bảo lưu. Bài thực hành thiết lập 0x60100000 (kernel), 0x64000000 (initramfs),
0x60000000 (DTB). Nếu chọn địa chỉ ngoài RAM, sai loại image hoặc chồng lấn vùng nhớ, bootz có thể từ chối image, hỏng DTB/initramfs hoặc hệ thống đứng/Kernel Panic.

Câu 3. Giải mã MIDR 0x410fc090

MIDR là thanh ghi định danh CPU: bits 31:24 = 0x41 (ARM implementer), bits 23:20 = 0x0 (variant), bits
19:16 = 0xF (architecture field), bits 15:4 = 0xC09 (part number – ARM Cortex-A9), bits 3:0 = 0x0 (revision). Đây là thông tin phần cứng phù hợp với đầu ra /proc/cpuinfo: implementer 0x41, CPU part 0xC09, variant 0 và revision 0.

9. Trả lời câu hỏi phân tích (tiếp)

Câu 4. Vì sao BusyBox cần static linking?

Rootfs initramfs rất nhỏ, không có sẵn dynamic loader và các shared library của glibc. Build BusyBox static khiến phần mã thư viện thiết yếu được liên kết vào ELF, giảm rủi ro thiếu phụ thuộc ngay khi chạy /init. Nếu build dynamic cho ARM hard-float, tối thiểu cần loader ld-linux-armhf.so.3 và libc.so.6; tùy các applet/cấu hình và phiên bản glibc có thể cần thêm libm.so.6, libresolv.so.2 hoặc libpthread.so.0 ở các hệ cũ. Không phải mọi file này đều bắt buộc với mọi binary; phải xác định chính xác bằng readelf -d hoặc ldd với đúng sysroot.

```
Ví dụ vị trí trên máy Ubuntu cài gcc-arm-linux-gnueabihf:
/usr/arm-linux-gnueabihf/lib/ld-linux-armhf.so.3
/usr/arm-linux-gnueabihf/lib/libc.so.6
/usr/arm-linux-gnueabihf/lib/libm.so.6
# Ki m tra đường d n thực t trên máy trước khi sao chép.
```

Câu 5. Bật SMP và so sánh -smp 1 với -smp 2

Ở chế độ SMP, kernel cần khởi tạo CPU thứ cấp, cơ chế đánh thức secondary cores, per-CPU data, scheduler runqueue trên từng CPU, inter-processor interrupts (IPI), cache-coherency, đồng bộ spinlock và phân phối công việc/IRQ. Với -smp 2, ảnh thực tế cho thấy /proc/cpuinfo có hai mục processor (0,1) và /sys/devices/system/cpu/online = 0-1.

Chỉ số	-smp 1	-smp 2
Số CPU online	Kỳ vọng 1 (chưa đo trong phiên gửi)	Thực tế: 2
/proc/cpuinfo processor	Kỳ vọng 1 mục (chưa đo)	Thực tế: 2 mục
Boot log SMP	Cần lưu log thực tế -smp 1	Kernel hỗ trợ SMP; uname: #1 SMP

Để có đủ yêu cầu quan sát hai dòng boot log thay đổi: cần chạy thêm cùng kernel với -smp 1, rồi trích log số CPU activated/online và so với phiên -smp 2. Không tự ghi dòng log chưa được thu thập.

10. Khó khăn gặp phải và cách khắc phục

Khó khăn thực tế	Cách xử lý / kết quả
BusyBox menuconfig báo thiếu ncurses	Kiểm tra thư viện/header ncurses; lựa chọn cấu hình bằng .config khi cần để tiếp tục build.
BusyBox networking/tc.c lỗi ký hiệu TCA_CBQ_*	Xung đột mã BusyBox cũ với header mới; vô hiệu hóa applet tc theo hướng dẫn để build BusyBox static.
Thiếu rootfs gây Kernel Panic	Bài thử đối chứng: chạy QEMU không có -initrd, thu ảnh lỗi VFS; thêm initramfs để boot thành công.
U-Boot saveenv thất bại	Flash báo địa chỉ không nằm ở sector boundary. Lưu lại log và giải thích rằng setenv vẫn có hiệu lực trong phiên hiện tại.

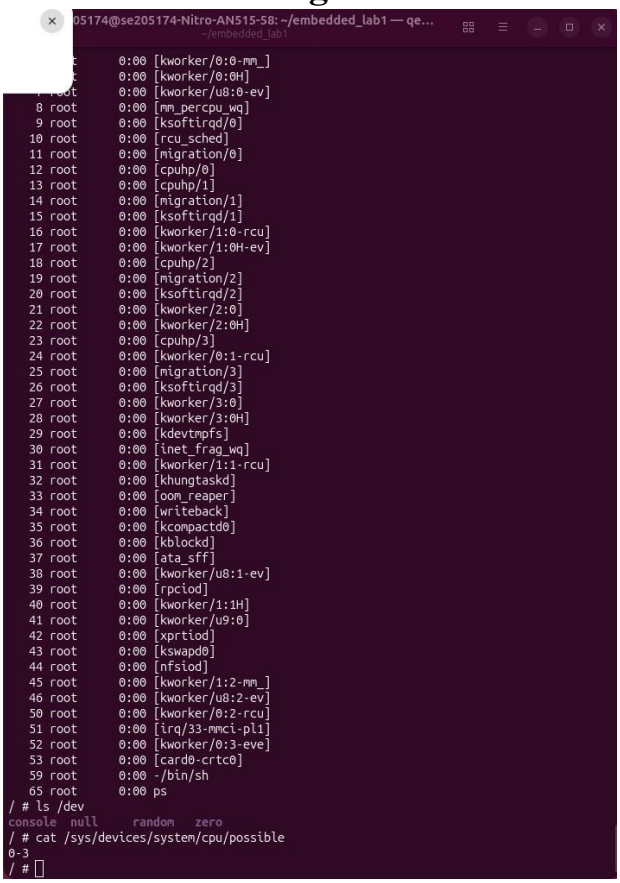
10.1. Phân biệt hai phương thức boot trong bài

Thử U-Boot bằng QEMU -kernel output/u-boot để tương tác U-Boot shell. Thử Linux bằng QEMU -kernel output/zImage -dtb ... -initrd ..., tức nạp kernel trực tiếp qua QEMU. Cả hai là yêu cầu thực hành hợp lệ nhưng không chứng minh U-Boot đã nạp kernel thành công trong cùng một chuỗi boot.

10.2. Đánh giá kết quả

Đã xác minh đủ sản phẩm Kernel, U-Boot, BusyBox static, initramfs và boot thành công đến shell. Kernel Panic đối chứng đã được ghi nhận; các lệnh uname, cpuinfo, meminfo, mount, ps và trạng thái CPU đã có ảnh thực tế. Những điểm còn cần tăng chất lượng minh chứng là ảnh đầu ra build gốc, log đầy đủ U-Boot dạng hình và phép so sánh -smp 1 với -smp 2.

11. Danh mục hình ảnh và minh chứng



Hình 4. BusyBox ps, danh sách /dev và trạng thái CPU possible trong QEMU.

Danh sách minh chứng hiện có trong bản PDF: Hình 1 – Kernel Panic; Hình 2 – boot và thông số hệ thống; Hình 3 – SMP online; Hình 4 – tiến trình BusyBox và thiết bị. Log U-Boot được trích từ Terminal thực tế trong Mục 4.